

Chapter 2 - Variables

With programming we can do arithmetics.

What is $3 + 4$?

It's 7!

Okay well let's test that. We can use print to make the number appear in our output console.

```
print(3 + 4)
```

--Output: 7

Run your code (meaning press F6 and then close the window to show the output) and your console should say 7.

Cool! Now what is $a + b$?

Uhm...

Well it could be anything. That's because "a" and "b" don't have a value. Let's change that.

```
a = 5
```

```
b = 3
```

Let's take another look, what is $a + b$? What we're really asking is "What is the value of a + the value of b?". In other words, what is $5 + 3$? Which is 8.

To prove that $a + b = 8$, we're going to print it.

```
a = 5
```

```
b = 3
```

```
print(a + b)
```

--Output: 8

Run your code again.

Here a and b are what we call *variables*. A variable is a word in which you can store a value. The number 3 is always 3, and 7 is always 7, but a variable can be anything you want it to be. Hence the name variable.

The word in which you store a value can be almost anything.

sheep = 3

test = 20

PANTS = 1040

asdfghjkl = 42

Variables are *case-sensitive*. That means that when you have the same word, but with different casing, it's not treated as the same. For example

sheep = 3

SHEEP = 10

sHeEp = 200

are three different variables, with each their own value.

You can do more than just summing up numbers.

$a = 20 - 10$ --*Subtraction*

$b = 20 * 10$ --*Multiplication*

$c = 20 / 10$ --*Division*

$d = 20 ^ 10$ --*Exponentiation*

For numbers with decimals we use a dot.

$a = 10.4$

$b = 2.63$

$c = 0.1$

$\pi = 3.141592$

Take a look at the following code:

$X = 5$

$Y = 3$

$Z = X + Y$

First we say $X = 5$. When we give a variable a value, we call that an *assignment*. We *assign* 5 to X, and 3 to Y. Next we assign $X + Y$ to Z. So now Z equals 8. Remember that you can

always check the value of a variable with print. If we were to change the value of X or Y after $Z = X + Y$, it would not affect Z. It would still be 8.

```
X = 5
```

```
Y = 3
```

```
Z = X + Y
```

```
X = 2
```

```
Y = 40
```

```
print(Z)
```

--Output: 8

This is because to the computer Z is not $X + Y$, it's simply 8.

Strings

A variable can also store text.

```
text = "Hello World!"
```

This is what we call a *string*. Because it's a string of characters.

We can connect strings by using two dots (..)

```
name = "Daniel"
```

```
age = "25"
```

```
text = "Hello, my name is " .. name .. ", and I'm " .. age .. " years old."
```

```
print(text)
```

--Output: "Hello, my name is Daniel, and I'm 25 years old."

Variable naming rules

There are a few rules when naming a variable. First of all, your variable may have a number in it, but not at the start.

```
test8 --Good
```

```
te8st --Good
```

8test --*Bad, error!*

Your variable name also can't include any special characters like @#\$%^&*.

And finally, your variable name can't be a keyword. A keyword is a word that the programming language uses. Here's a list of keywords:

and break do else elseif

end false for function if

in local nil not or

repeat return then true until while

Usage

Variables can be used to keep track of things. For example, we can have the variable coins, and every time we pick up a coin we can do `coins = coins + 1`.

Summary

Variables are words in which we can store a value like a number or text. You can name them whatever you want, with a few exceptions. Variables are case-sensitive.